🧠 ML Deployment Workshop: From Training to Model Drift

Duration: 4 hours | **Audience:** Engineers new to ML | **Format:** Live coding + guided lab

🚀 Workshop Overview

This workshop takes you through the complete ML deployment lifecycle, from training a simple CNN to detecting model drift in production. You'll get hands-on experience with real deployment challenges.

📋 Prerequisites & Setup

Local Environment (Colab/Jupyter)

```
# Install required packages
pip install torch torchvision scikit-learn numpy matplotlib seaborn
pip install fastapi uvicorn python-multipart pillow requests joblib
```

EC2 Setup (Instructor Pre-setup)

- **Instance Type:** t2.medium or t3.medium
- **AMI:** Ubuntu 20.04 LTS
- **Security Group:** Allow SSH (22) and Custom TCP (8000)
- **Storage:** 20GB minimum

📂 Project Structure

```
ml_deployment_workshop/
├── train_model.py          # Part 2: Training script
├── app.py                  # Part 3: FastAPI server
├── requirements.txt        # Dependencies
├── test_client.py         # Testing script
├── simulate_drift.py      # Part 5: Drift simulation
├── deploy_to_ec2.sh       # Deployment commands
├── saved_models/          # Model artifacts
```

```
├── data/                    # Dataset cache
└── plots/                   # Generated plots
```

📌 Part 1: Setup & Verification (15 min)

Step 1.1: Clone and Setup

```
# Create project directory
mkdir ml_deployment_workshop
cd ml_deployment_workshop

# Clone this git repository
git clone https://github.com/InfinitelyAsymptotic/ik.git
git checkout mlDeploymentWorkshop

# Verify Python environment
python --version  # Should be 3.8+
pip install -r requirements.txt
```

Step 1.2: Test EC2 Connection

```
# Test SSH connection (replace with your details)
ssh -i your-key.pem ubuntu@your-ec2-ip

# On EC2, verify Python
python3 --version
sudo apt update
sudo apt install -y python3-pip python3-venv
```

📌 Part 2: Train a Simple CNN (30 min)

Learning Objectives

- Understand basic ML training pipeline

- Learn model serialization best practices
- Create reproducible training workflows

Step 2.1: Run Training Script

```
python train_model.py
```

What happens:

1. Downloads CIFAR-10 dataset (~170MB)
2. Trains a simple CNN (5 epochs, ~5 minutes)
3. Evaluates on test set
4. Saves model + metadata to `saved_models/`
5. Generates training plots

Step 2.2: Inspect Results

```
ls saved_models/
# Should contain:
# - model.pth (PyTorch state dict)
# - model_metadata.joblib (model info)

ls plots/
# Should contain:
# - training_progress.png
```

🎯 Key Takeaways

- **Model Versioning:** Always save metadata with models
- **Reproducibility:** Fixed random seeds, documented preprocessing
- **Monitoring:** Track training metrics from day one

📌 Part 3: Build FastAPI Inference Server (30 min)

Learning Objectives

- Create production-ready ML APIs

- Handle image preprocessing pipelines
- Implement proper error handling

Step 3.1: Start the API Server

```
# Start locally first
python app.py

# Or with uvicorn directly
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

Step 3.2: Test the API

```
# In another terminal
python test_client.py
```

API Endpoints:

- `GET /ping` - Health check
- `GET /model-info` - Model metadata
- `POST /predict` - Image prediction (base64)
- `POST /predict-file` - Image prediction (file upload)

Step 3.3: Manual Testing

```
# Test health endpoint
curl http://localhost:8000/ping

# Test model info
curl http://localhost:8000/model-info
```

🎯 Key Takeaways

- **Input Validation:** Always validate API inputs
- **Preprocessing Consistency:** Match training preprocessing exactly
- **Error Handling:** Graceful degradation for bad inputs
- **Monitoring:** Log all predictions for later analysis

📌 Part 4: Deploy to EC2 (30 min)

Learning Objectives

- Understand cloud deployment workflows
- Configure security groups and networking
- Debug deployment issues

Step 4.1: Package for Deployment

```
# Create deployment package
mkdir deployment_package
cp app.py deployment_package/
cp requirements.txt deployment_package/
cp -r saved_models deployment_package/
```

Step 4.2: Deploy to EC2

```
# Copy files to EC2
scp -i your-key.pem -r deployment_package ubuntu@your-ec2-ip:~/ml_deployment

# SSH into EC2
ssh -i your-key.pem ubuntu@your-ec2-ip

# On EC2: Set up environment
cd ml_deployment
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# Start the service
uvicorn app:app --host 0.0.0.0 --port 8000
```

Step 4.3: Configure Security & Test

1. **Security Group:** Add inbound rule for port 8000

2. **Test Deployment:**

```
# From your local machine
curl http://your-ec2-ip:8000/ping

# Update test_client.py with EC2 IP and run
python test_client.py
```

🎯 Key Takeaways

- **Environment Consistency:** Virtual environments prevent conflicts
- **Security:** Never expose unnecessary ports
- **Process Management:** Use process managers (systemd/supervisor) in production
- **Monitoring:** Always monitor deployed services

📌 Part 5: Simulate Model Drift (30 min)

Learning Objectives

- Understand different types of model drift
- Implement drift detection strategies
- Plan model retraining workflows

Step 5.1: Run Drift Simulation

```
# Make sure your API is running (local or EC2)
python simulate_drift.py
```

Drift Scenarios:

1. **Baseline:** Original CIFAR-10 test data
2. **Quality Drift:** Noisy CIFAR-10 images
3. **Severe Quality Drift:** Very noisy images
4. **Domain Drift:** CIFAR-100 images (different domain)

Step 5.2: Analyze Results

The script generates:

- `plots/drift_analysis.png` - Performance comparison charts
- `drift_report.txt` - Automated drift analysis
- `performance_log.json` - Detailed metrics

🎯 Key Takeaways

- **Data Quality Monitoring:** Track input data statistics
- **Performance Monitoring:** Log accuracy, confidence, latency
- **Threshold Setting:** Define acceptable performance ranges
- **Retraining Triggers:** Automate model refresh workflows

📌 Bonus: Real-World Gotchas (15 min)

Common Production Issues

1. Preprocessing Mismatches

```
# Training: PIL Image → Tensor → Normalize
# Inference: Base64 → PIL → Tensor → Normalize
# Issue: Different image formats, color channels
```

2. Model Versioning

```
# Good: Version your models
saved_models/
├── v1_2024_01_15/
│   ├── model.pth
│   └── metadata.joblib
└── v2_2024_02_01/
    ├── model.pth
    └── metadata.joblib
```

3. Performance Monitoring

```
# Log everything for analysis
{
    "timestamp": "2024-01-15T10:30:00Z",
    "model_version": "v1.0",
    "prediction": "cat",
    "confidence": 0.85,
    "latency_ms": 45,
    "input_hash": "abc123"
}
```

4. Rollback Strategy

```
# Always have a rollback plan
if current_model_accuracy < threshold:
    switch_to_previous_model()
    trigger_retraining_pipeline()
```

🛠️ Troubleshooting Guide

Common Issues

1. Port 8000 already in use

```
# Find and kill process
lsof -i :8000
kill -9 <PID>
```

2. CUDA out of memory

```
# In train_model.py, reduce batch size
trainloader = DataLoader(trainset, batch_size=32)  # Reduce from 64
```

3. EC2 connection refused

- Check security group (port 8000 open)

- Verify API is running with `--host 0.0.0.0`
- Check EC2 logs: `tail -f app.log`

4. Model loading errors

```
# Ensure model architecture matches exactly
model = SimpleCNN(num_classes=10)  # Must match training
```

🎯 Workshop Learning Outcomes

By the end of this workshop, you'll understand:

✅ **Training Pipeline:** Reproducible model training and validation
✅ **API Development:** Production-ready ML inference services
✅ **Cloud Deployment:** End-to-end deployment workflows
✅ **Drift Detection:** Monitoring and maintaining model performance
✅ **Production Challenges:** Real-world deployment gotchas